ResumeLens

Formal Language-Based Résumé Screening: Regular Expressions, Transducers, Finite Automata and Context-Free Grammars

Alejandro Rendón Garzón · Juan Diego Balanta Molina · Edwar Estacio — Universidad Icesi · Computación y Estructuras Discretas III · 2026-2

1. Problem

Recruiters screen hundreds of résumés written in different formats. The same qualification appears with many names: `JS`, `Javascript`, `JavaScript`; `sklearn`, `Scikit-learn`.

Many screening tools rely on statistical models whose decisions are hard to explain or audit.

Our question: do the qualifications *explicitly written* in a résumé satisfy a formally defined profile pattern? Each answer is a membership test `w ∈ L(M)`, traceable step by step.

Out of scope: ResumeLens does not rank candidates or make hiring decisions.

2. Approach: one formal model per stage

Stage	Formal model	Tool
1. Extraction	Regular expressions	`re`
2. Normalization	Finite-state transducers M = (Q, Σ, Γ, δ, ω, q_0, F)	pyformlang
2b. Sorting	Canonical slot order per profile	Python
3. Recognition	ε-NFA → minimal DFA, M = (Q, Σ, δ, q_0, F)	pyformlang
4. Candidate language	Context-free grammar G = (V, Σ, R, S), EBNF	textX
4b. Visualization	Validated model → HTML / Markdown	Python

Four profiles share one engine: **Full Stack Developer**, **Machine Learning Engineer** (given), **DevOps Engineer** and **Data Engineer** (team-defined).

3. Pipeline on the assignment example (Wednesday Addams)

Résumé text
"Technical Skills: JS, React.js, NodeJS, Postgres, Git."

→

1 · Regex
raw skills: JS, React.js, NodeJS, Postgres, Git

→

2 · FST
JAVASCRIPT, REACT, NODE_JS, POSTGRESQL, GIT

→

2b · Sort
filter + slot order for each profile

→

3 · Automata
FULL_STACK: **ACCEPTED**
ML / DevOps / Data: **REJECTED**

→

4 · CFG
validated DSL → HTML report

4. Normalization with transducers

- One transducer per canonical token, generated from the vocabulary (30 tokens): `react.js`, `reactjs`, `react js` → `REACT`.
- Input is preprocessed (strip, lower case); Σ = a–z, 0–9, space, `.` `-` `/`; Γ = A–Z, `_`.
- Whole-string matching: `java` never matches the prefix of `javascript`.
- No accepting path → the string goes to *unrecognized* (e.g. Figma, Jira); nothing is guessed.

Candidate language (CFG)

- Blocks `contact`, `education*`, `experience*`, `skills`, `classification` with nested braces need a context-free grammar.
- Invalid profiles (unknown block, verdict `MAYBE`, unclosed string) are rejected with line and column.

5. Profile patterns as finite automata

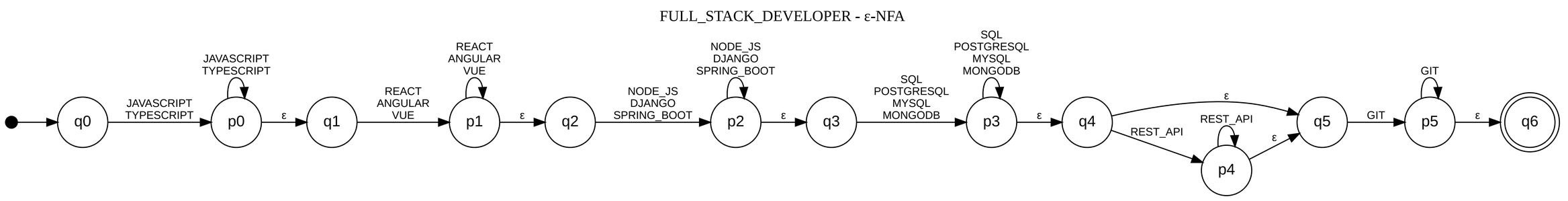
A profile is an ordered list of slots $S_1 \dots S_n$. A mandatory slot needs one or more of its tokens; an optional slot may be absent:

$$L(P) = S_1^{k_1}\, S_2^{k_2} \cdots S_n^{k_n}, \quad k_i \in \{+, *\}$$

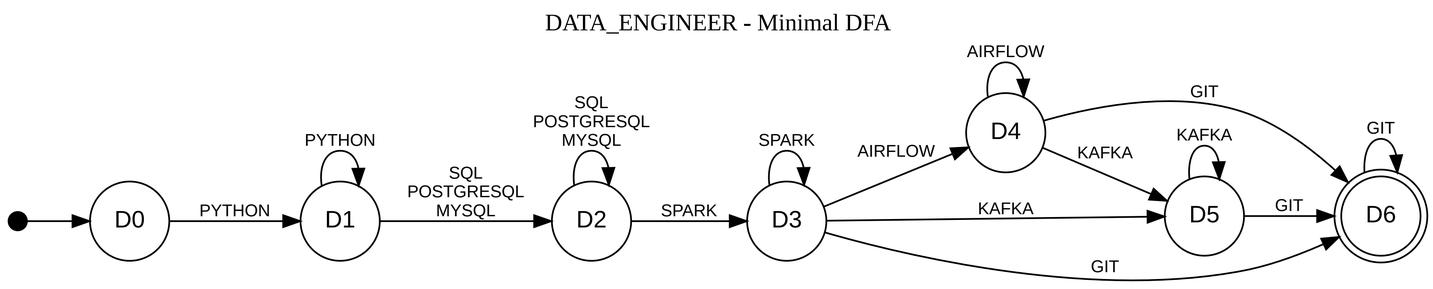
- Built as an **ε-NFA**: states q_i (before slot i) and p_i (slot i has a token); ε-edges leave a slot or skip an optional one.
- Subset construction + minimization give an equivalent **minimal DFA** (7 states per profile); equivalence is checked in the tests.
- Tokens outside the profile are filtered first, so extra skills never cause a rejection.

6. Example automata

Full Stack Developer — ε-NFA (REST API slot optional)



Data Engineer — minimal DFA (two consecutive optional slots: Airflow, Kafka)



7. Results

Sample résumé	Accepted profile
Wednesday Addams	**Full Stack Developer**
Mary Jane Watson	**Machine Learning Engineer**
Linus Torvalds	**DevOps Engineer**
Grace Hopper	**Data Engineer**
Rejected candidate (Figma, Photoshop, Jira, Git)	**none**

- **249 automated tests** pass (pytest).
- Same verdicts under any permutation of the skills list.
- Adding a skill outside a profile never changes that profile's verdict.
- One résumé can satisfy several profiles (e.g. ML + Data Engineer).

8. Discussion and limitations

- **Explainable:** every verdict can be traced to a regex match, a transducer path and an automaton run.
- **Presence, not proficiency:** the system checks that a qualification is written, not its level or context.
- **Closed vocabulary:** unknown technologies are reported, not matched, until the vocabulary is extended.
- **Human choices remain:** profiles, slots and aliases are designed by people and can carry bias.
- **Future work:** context-sensitive rules (e.g. minimum years per technology) with attribute grammars or counters.

Key references

Hopcroft, Motwani & Ullman (2006) · Mohri (1997) *Computational Linguistics* 23(2) · Chiticariu et al. (2013) EMNLP · Dejanović et al. (2017) textX · Romero (2021) pyformlang · Raghavan et al. (2020) FAT*

Code and documentation: github.com/AlejoRendon095/ResumeLens_IntegrativeTask